

OpenDccCLib — Architecture (As-Built)

Generated 2026-09-25 from the repository sources by `documentation/guides/build_pdfs.py`

Table of Contents

1. Overview	3
2. Feature flags	4
3. User-configurable constants	5
4. The <code>dcc_config_t</code> struct	6
5. Public API surface	7
Lifecycle & ISR entry points (<code>dcc_config.h</code>)	7
Application modules (role-first naming)	7
6. Internal modules	9
7. Execution contexts	11
8. Key invariants	12

Verified against the source at 2026-09-25 (commit 65d4570, followed by a full documentation audit against src/dcc). This document describes the library as it is implemented today. For the original design intent (which predates the role-first naming refactor), see archive/OpenDccCLib_Requirements.md. For what is done vs. pending, see ComplianceOverview.md.

1. Overview

OpenDccCLib is a portable C library implementing NMRA DCC (S-9.1 through S-9.3.2) for both command-station and decoder roles. Design constraints:

- **No dynamic memory.** All buffers and state are statically sized by `USER_DEFINED_DCC_*` constants.
- **No OS / RTOS.** Runs from a main loop plus a small number of ISRs.
- **Dependency injection everywhere.** Cross-module calls go through `interface_dcc_*_t` function-pointer structs, never direct includes; the utilities modules are the documented exemption (`dcc_railcom_utilities` is called directly by the two RailCom modules). `dcc_config.c` (the wiring module) includes the headers of every module it wires.
- **Portable hardware layer.** The library never touches hardware directly; the user supplies driver callbacks in one `dcc_config_t` struct.

The user includes `dcc_config.h`, populates one struct, and calls `DccConfig_initialize()`; the application API comes from the `dcc_application_*.h` headers.

2. Feature flags

Defined by the user in `dcc_user_config.h`, validated with `#error/#warning` in `dcc_config.h` and `dcc_types.h`.

Flag	Enables
<code>DCC_COMPILE_COMMAND_STATION</code>	Packet encoding, bit framing, scheduler, service-mode core; with <code>DCC_COMPILE_RAILCOM</code> the RailCom cutout + receive
<code>DCC_COMPILE_DECODER</code>	Packet decoding, CV storage, fail-safe; with <code>DCC_COMPILE_RAILCOM</code> the RailCom transmit engine
<code>DCC_COMPILE_ACCESSORY_DECODER</code>	Accessory-decoder RailCom reply helpers (SRQ, status, time, error); needs <code>DCC_COMPILE_RAILCOM</code> , and packet reception needs <code>DCC_COMPILE_DECODER</code>
<code>DCC_COMPILE_RAILCOM</code>	Every RailCom module, config field and function; must be paired with a role flag (<code>dcc_types.h</code>)
<code>DCC_COMPILE_SERVICE_MODE_DIRECT</code>	Direct byte/bit programming (requires CS)
<code>DCC_COMPILE_SERVICE_MODE_PAGED</code>	Paged programming (requires CS)
<code>DCC_COMPILE_SERVICE_MODE_REGISTER</code>	Physical register programming (requires CS)
<code>DCC_COMPILE_SERVICE_MODE_ADDRESS</code>	Address-only programming (requires CS)
<code>DCC_COMPILE_SERVICE_MODE_TASK_{DIRECT, PAGED, REGISTER, ADDRESS, DETECT}</code>	The read/write/verify orchestrators and mode detection on top of the primitives; not cross-checked against their primitive

RailCom and each service-mode method are compile-time choices; hardware is optional at runtime. `DCC_COMPILE_RAILCOM` strips every RailCom module, field and function. The four primitive flags each require `DCC_COMPILE_COMMAND_STATION` (`#error`); the `TASK_*` flags are not cross-checked. Within a compiled feature, missing hardware is tolerated: RailCom stays inert until `main_track.railcom` and `railcom_timer_start` are wired, and decoder transmit until `railcom_tx_pin_set` is. Service mode without `current_sense_read` still transmits and finishes `NO_ACK` or `ERROR`. A device may define both CS and `DECODER` for booster/repeater use; `dcc_config.h` warns when no role is enabled.

3. User-configurable constants

Set in `dcc_user_config.h`:

- `USER_DEFINED_DCC_SCHEDULER_SLOT_COUNT` — max concurrent active scheduler slots
- `USER_DEFINED_DCC_MAX_LOCOMS` — locomotives tracked by the application's loco table
- `USER_DEFINED_DCC_PREAMBLE_BITS_OPS` — operations-mode preamble (≥ 14 ; ≥ 16 with RailCom)
- `USER_DEFINED_DCC_RAILCOM_BUFFER_DEPTH` — RailCom receive ring-buffer depth
- `USER_DEFINED_DCC_SERVICE_MODE_RETRIES`
- `USER_DEFINED_DCC_ACK_THRESHOLD_MA`, `..._ACK_MIN_DURATION_US`, `..._ACK_MAX_DURATION_US`; `..._ACK_DROP_OUT_TOLERANCE_US` is optional (default 116)
- `USER_DEFINED_DCC_DECODER_MAX_FUNCTIONS` (required, ≥ 1 , but not used by the library: F0–F68 are always dispatched), `..._DECODER_PACKET_QUEUE_DEPTH` (≥ 2 , one slot reserved; a full queue drops the newest packet)

Every count is validated ≥ 1 ; the command-station constants are required only with `DCC_COMPILE_COMMAND_STATION`; a preamble of 16–17 with RailCom raises a `#warning`. The refresh pacing constants `DCC_REFRESH_PROMPT_SENDS`, `DCC_REFRESH_COLD_CYCLES`, `DCC_REFRESH_COLD_MAX_CYCLES` and `DCC_REFRESH_CV11_FLOOR` are `#ifndef` defaults in `dcc_defines.h` that `dcc_user_config.h` may override, with `#error` guards on their relationships.

One-shot packet repeat counts are not user constants: the builders set them from the `DCC_REPEAT_*` table in `dcc_defines.h` (CV write 2, verify 1, date 3, time 1, accessory NOP/stop 1, everything else 2); an application may overwrite `repeat_count` after a builder returns, and 0 means the scheduler never sends the packet.

4. The `dcc_config_t` struct

One user-facing struct (`dcc_config.h`). Required fields must be non-NULL; NULL-optional fields disable the associated feature at runtime.

Common (always required): `lock_shared_resources`, `unlock_shared_resources`, `get_timestamp_usec`.

Command Station: a shared fixed-period DCC timer (`shared_timer_start/stop`, clocked at `DCC_ONE_BIT_HALF_PERIOD_US = 58 µs`). Per-channel hardware is described by `dcc_output_hw_t` for `main_track` and `service_track`: `pin_toggle`, `track_power_set`, optional `current_sense_read`, an optional nested `dcc_railcom_hw_t`, and `timer_start/stop`, which the header marks REQUIRED but `dcc_config.c` never calls (it substitutes shared-timer wrappers). Of these the library reads `main_track.track_power_set`, `service_track.current_sense_read` (ACK sampling) and `main_track.railcom`; `service_track.track_power_set`, `main_track.current_sense_read` and `service_track.railcom` are not wired in this release. With `DCC_COMPILE_RAILCOM`: a RailCom one-shot timer (`railcom_timer_start/stop`), the five cutout periods (`railcom_cutout_start_delay_us`, `railcom_uart_rx_delay_us`, `railcom_ch1_window_us`, `railcom_ch1_ch2_gap_us`, `railcom_ch2_window_us`; 0 = spec default), and `dcc_railcom_hw_t` (`begin_railcom_cutout`, `end_railcom_cutout`, `uart_rx_enable`, `uart_rx_disable`, `uart_read`, `on_railcom_datagram_result`). Optional callbacks: `on_packet_sent` and `on_accessory_srq`; the latter is never invoked in this release and only enables the 100 ms accessory polling. Service-mode results are delivered per call through the task callbacks (`on_complete`, `on_progress`, `on_detect`), not through a config callback.

Decoder: required `cv_read/cv_write/cv29_apply_supported_features` (the last tolerates NULL); NULL-optional `factory_reset`, `cv_read_indexed/cv_write_indexed`, `start_ack_pulse/stop_ack_pulse`, and with `DCC_COMPILE_RAILCOM` `railcom_tx_pin_set + railcom_delay_us` (required once the pin is set) + `on_railcom_request`, which runs on the edge path rather than the main loop. Twelve `on*_command` notification callbacks (`speed`, `emergency_stop`, `function`, `accessory_basic`, `accessory_extended`, `cv_write`, `cv_verify`, `cv_bit`, `consist`, `binary_state_short`, `binary_state_long`, `analog_function`) plus `on_failsafe_entered/exited`. The DCC edge interrupt is masked during a RailCom transmit through `lock_shared_resources`; there is no separate edge-IRQ hook. Note the CV callbacks (`on_cv_write_command`, `on_cv_verify_command`, `on_cv_bit_command`) carry a `bool service_mode` argument so the application can distinguish service-mode programming from POM.

5. Public API surface

Lifecycle & ISR entry points (`dcc_config.h`)

- `DccConfig_initialize(const dcc_config_t *)`
- `DccConfig_run()` — main loop; **all application callbacks fire from here, except the decoder's `on_railcom_request`, which runs on the edge path**
- `DccConfig_58us_timer_isr()` — [CS] shared bit timer
- `DccConfig_railcom_oneshot_timer_isr()` — [CS + RAILCOM] RailCom cutout state machine
- `DccConfig_set_railcom_cutout_timing()`, `DccConfig_cancel_railcom_cutout()`, `DccConfig_railcom_cutout_is_active()` — [CS + RAILCOM] runtime cutout control
- `DccConfig_100ms_timer_tick()` — [CS] RailCom accessory polling only (an accessory packet every 5 s when `main_track.railcom` and `on_accessory_srq` are both set); it handles no timeouts
- `DccConfig_decoder_edge_isr(uint32_t timestamp_usec)` — [DECODER] input-capture edge

Application modules (role-first naming)

Module	Role	Public prefix
<code>dcc_application_command_station_main_track</code>	CS	<code>DccApplicationCommandStationMainTrack_</code> — <code>power_on/off</code> , <code>send_packet</code> , <code>add_to_auto_refresh</code> , <code>remove_from_auto_refresh</code> , <code>remove_all_auto_refresh</code>
<code>dcc_application_command_station_service_track</code>	CS	<code>DccApplicationCommandStationServiceTrack_</code> — <code>power_on/off</code> , <code>enter/exit_service_mode</code> , <code>is_service_mode_active</code> ; <code>direct_{read,write}_{cv,bit}</code> , <code>paged_{read,write}_{cv,bit}</code> , <code>register_{read,write}_{cv,bit}</code> , <code>register_verify_value</code> , <code>register_factory_reset</code> , <code>address_{read,write,verify}</code> , <code>address_{read,write}_bit</code> , <code>detect_mode</code> , each group gated by its <code>DCC_COMPILE_SERVICE_MODE_TASK_*</code> flag
<code>dcc_application_command_station_packet</code>	CS	<code>DccApplicationCommandStationPacket_load_*</code> — packet builders
<code>dcc_application_decoder_cv</code>	DECODER	<code>DccApplicationDecoderCv_</code> — <code>read</code> , <code>write</code> , <code>is_locked</code>
<code>dcc_application_decoder_railcom</code>	RAILCOM + (DECODER or ACCESSORY)	<code>DccApplicationDecoderRailcom_send_*</code> — address feedback, POM response, dynamic data, ack/nack, track search, cv auto transfer, raw
<code>dcc_application_accessory_decoder_railcom</code>	RAILCOM + ACCESSORY	<code>DccApplicationAccessoryDecoderRailcom_</code> — <code>get_srq_state</code> , <code>SRQ</code> , status (1/4/extended), time/error report, cutout/stop hooks

Not wired in this release: `DccConfig_initialize()` does not call `DccApplicationDecoderRailcom_initialize` or `DccApplicationAccessoryDecoderRailcom_initialize`, although their headers say it does. Without an interface their calls transmit nothing, and the transmit engine has no sink for them; decoders answer through `on_railcom_request`. The decoder CV module is wired (onto `dcc_cv_storage`, so the lock, CV 29 filter and CV 8 reset apply).

Migration note: the pre-refactor modules `dcc_application_main_track` and `dcc_application_service_track` still exist and are still compiled/tested alongside the role-first modules above; `dcc_config.c` does not wire them. Retiring them is tracked in `ComplianceOverview.md`. The third one, `dcc_packet_encoder`, was removed on 2026-09-23: it was an uncalled duplicate of `dcc_application_command_station_packet` and had drifted (it still carried the `repeat_count = 0` "never transmitted" defect fixed in the live module).

6. Internal modules

Module	Role	Responsibility
dcc_conf ig	always	Wiring: builds the interface structs of the modules it wires and owns their contexts (the service-mode task modules and the decoder-side modules keep their own static state), <code>initialize()</code> / <code>run()</code> /ISR dispatch
dcc_type s	always	Typedefs, user-constant validation
dcc_defi nes	always	Protocol constants: timing, instruction masks, CV numbers, RailCom IDs
dcc_sche duler	CS	Priority queue, duplicate combining, paced auto-refresh: a changed slot is sent <code>DCC_REFRESH_PROMPT_SENDS</code> times at full rate, then kept alive every <code>DCC_REFRESH_COLD_CYCLES</code> packet cycles and never later than <code>DCC_REFRESH_COLD_MAX_CYCLES</code> (selection order overdue, in-burst, due; <code>DCC_REFRESH_COLD_CYCLES = 0</code> is the flat round-robin ring); one-shots are sent <code>repeat_count</code> times (the builders set the <code>DCC_REPEAT_*</code> defaults; 0 = never sent, and in this release never freed). Pending one-shots always go before refresh slots and priority ranks only one-shots; the prompt burst is re-armed on every insert; an idle spacer separates back-to-back packets to the same short address 112–127
dcc_bit_ encoder	CS	ISR bit framing from the shared fixed-period timer
dcc_rail com_cuto ut	CS + RAILCOM	RailCom cutout timer state machine
dcc_rail com_comm and_stat ion	CS + RAILCOM	Receive drain after each cutout, Ch1/Ch2 datagram assembly (split by byte count), receive ring, tagging with loco addresses
dcc_rail com_util ities	RAILCOM, any role	4/8 code words (S-9.3.2 Table 2): encode for the decoder roles, decode for the command station
dcc_serv ice_mode _common	CS	Shared ACK detection, reset sequencing, retry
dcc_serv ice_mode _{direct ,paged,r egister, address}	CS	Per-mode programming primitives
dcc_serv ice_mode _task_{d irect,pa ged,regi ster,add ress,det ect}	CS + own TASK_ * flag	Read/write/verify orchestration on the primitives; mode detection
dcc_bit_ decoder	DECODER	Edge-timestamp → bit classification → byte assembly

Module	Role	Responsibility
dcc_packet_decoder	DECODER	Parse bytes → structured commands, XOR, address match, deferred dispatch queue
dcc_cv_storage	DECODER	CV abstraction, decoder lock, factory reset, indexed CVs, CV29 feature mask
dcc_fail_safe	DECODER	S-9.2.4 packet time-out (CV11 in 100 ms units)
dcc_railcom_decoder	DECODER + RAILCOM	RailCom transmit engine (bit-bang) and reply arming

Most modules own an `interface_dcc_<module>_t` of function pointers, populated by `dcc_config.c` (`dcc_railcom_utilities` and the packet builders have none, and the two RailCom application modules and two legacy modules noted in §5 are not populated). This makes every dependency mockable in unit tests and lets an MCU swap touch only the config struct.

7. Execution contexts

Three contexts, with a deliberately small locked region:

- **ISR (timer / edge):** bit encoder advances the bit state machine and toggles the output pin; the 58 μ s tick also samples service-track current for ACK detection; RailCom cutout timing fires here. The decoder edge path classifies bits, assembles bytes, queues the finished packet, calls `on_railcom_request`, and bit-bangs the RailCom reply at 4 μ s intervals with interrupts masked (about 454 μ s; the CPU is otherwise idle then). ISR work is otherwise kept minimal.
- **Main loop (`DccConfig_run()`):** scheduler selection, packet build, service-mode state machine, RailCom receive drain, decoded-packet dispatch, fail-safe timing, 6 ms ACK-pulse timing. **Every command callback fires from this context; the decoder's `on_railcom_request` is the one exception.**
- **User API (any thread):** the `DccApplication*` calls write into shared state; on threaded hosts they must be protected by the user's `lock/unlock`.

The handoff between the ISR-level bit encoder and the main-loop scheduler is single-buffered: the encoder holds one active packet and a `packet_loaded` flag; the ISR clears the flag at the end bit and the main loop loads the next packet without taking a lock (the only lock calls in the library surround the decoder's RailCom transmit). The cutout arm is deferred one tick so it lands on the end bit's last edge, because the encoder's state machine runs a half-bit ahead of the wire.

8. Key invariants

1. No direct cross-module includes in the core library — only via interface structs; the utilities modules are exempt (style guide).
2. Every new function/type is wrapped in the appropriate `DCC_COMPILE_*` guard.
3. No dynamic memory; all sizing comes from `USER_DEFINED_DCC_*` constants.
4. CV numbers are 1-based everywhere; the 0-based wire encoding is confined to the packet builders (`dcc_application_command_station_packet.c`), the service-mode primitives (`dcc_service_mode_direct.c`, `dcc_service_mode_paged.c`), and the packet decoder's conversion back.
5. Callbacks are main-loop only, except `on_railcom_request`; ISRs do pin/flag work and defer notification.